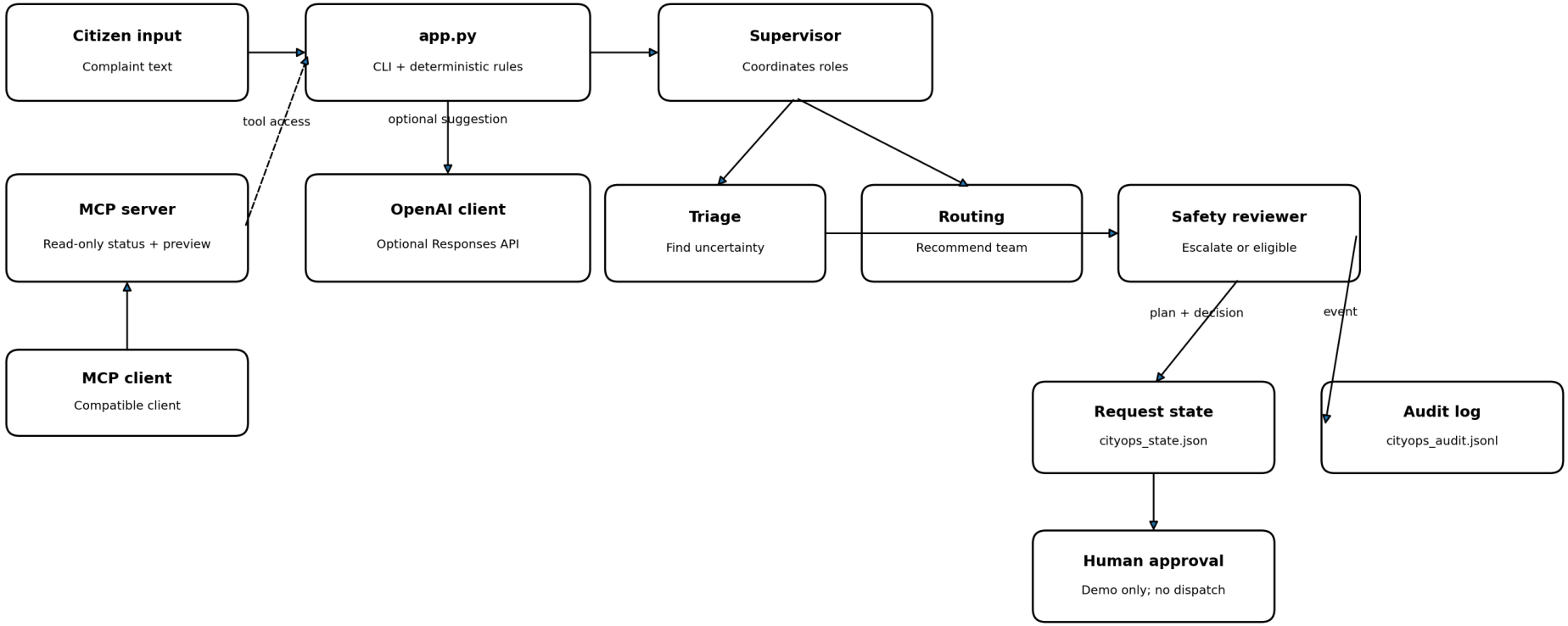
CityOps AI — GH-600 Workshop Architecture

Runtime workflow, tool integration, state/audit, and GitHub engineering lifecycle

CityOps AI — GH-600 Workshop Architecture

Runtime workflow and GitHub engineering lifecycle



GitHub development and governance lifecycle



Purpose and runtime flow

CityOps AI is a learning-only municipal service request planning assistant. It does not connect to a real municipal dispatch system.

1. A complaint enters app.py and is classified with deterministic rules.

2. openai_client.py optionally obtains a suggestion from the OpenAI Responses API. The API key is loaded from local .env; output is validated.

3. A supervisor coordinates triage, routing, and safety-review functions.

4. Request state is stored in cityops_state.json and events are appended to cityops_audit.jsonl.

5. Unknown or review-required requests are escalated. Eligible requests may be marked approved by a demonstration CLI command; no real dispatch occurs.

6. mcp_server.py exposes read-only request-status and routing-preview tools to compatible MCP clients.

Trust boundaries and safety

• Complaint text is untrusted input. Model output is a suggestion, not authorization.

• Unknown and review-required requests cannot be approved through the application guard.

• CI tests and evaluation run without a live model or API key.

• .env, local state, and audit logs must remain out of Git.

• The CLI approver name is not identity verification; the local JSONL audit file is not tamper-proof.

• GitHub security feature availability depends on plan, repository visibility, and organization policy.

Editable sources

architecture.mmd is the editable Mermaid diagram source. architecture.yaml captures the architecture as structured data. architecture.md documents responsibilities and trust boundaries.